Hand Swipe Direction Detection App

System Architecture — Class Project

SYSTEM ARCHITECTURE GENERATION PROMPT

Hand Swipe Direction Detection — Class Project

Create a complete but simple and beginner-friendly **System Architecture** for a Python-based computer vision class project called:

“Hand Swipe Direction Detection App”

This is a **small classroom project**, not a production/client-level application. The architecture should be practical, understandable, and easy for a student to implement and explain.

The main purpose of the project is to detect the **direction in which a user's hand moves/swipes** using a webcam.

The system should detect:

• SWIPE LEFT

• SWIPE RIGHT

• SWIPE UP

• SWIPE DOWN

Optionally:

• NO SWIPE / IDLE

• UNKNOWN

---

1. IMPORTANT PROJECT DEFINITION

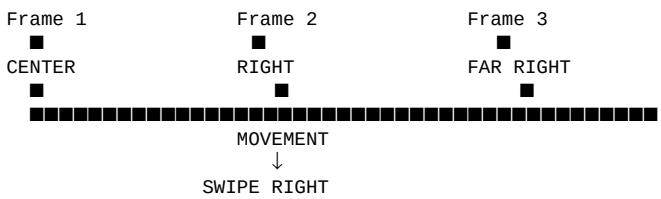
This project is **NOT simply detecting the current position of the hand**.

For example:

```
Hand is currently on the left side
→ This does NOT automatically mean LEFT.
```

Instead, the system must analyze **movement over time**.

Example:



Therefore, the system must compare the hand's position between multiple video frames.

The core concept is:

```
Previous Hand Position
```

```
          ↓
      Movement
          ↓
Current Hand Position
          ↓
Calculate ΔX / ΔY
          ↓
Determine Direction
          ↓
SWIPE LEFT / RIGHT / UP / DOWN
```

---

2. PROJECT OBJECTIVE

The application should:

1. Open the webcam.
2. Capture live video frames.
3. Detect a hand.
4. Track the hand's position across multiple frames.
5. Calculate the movement between previous and current positions.
6. Determine the dominant movement direction.
7. Detect a swipe/gesture only when sufficient movement occurs.
8. Display the detected swipe direction on the screen.
9. Handle common errors safely.
10. Allow the user to exit using `Q`.

Example:

```
User moves hand:

← ← ← ←

System detects:

SWIPE LEFT
```

Another example:

```
User moves hand:

→ → → →

System detects:

SWIPE RIGHT
```

---

3. CORE CONCEPT — TEMPORAL MOVEMENT DETECTION

Explain that the project is based on **movement over time**, not static position.

The system should maintain at least:

```
Previous Position
Current Position
```

For example:

```
Previous:
```

```
(x1, y1)

Current:
(x2, y2)
```

Calculate:

```
ΔX = x2 - x1
ΔY = y2 - y1
```

Then determine the dominant movement.

For example:

```
If |ΔX| > |ΔY|:

    ΔX > threshold
        → SWIPE RIGHT

    ΔX < -threshold
        → SWIPE LEFT
```

And:

```
If |ΔY| > |ΔX|:

    ΔY > threshold
        → SWIPE DOWN

    ΔY < -threshold
        → SWIPE UP
```

Explain this clearly and visually.

---

4. SWIPE DETECTION VS STATIC POSITION

Clearly explain the difference:

Static Position Detection

```
Hand is on the left side
→ LEFT
```

Swipe Detection

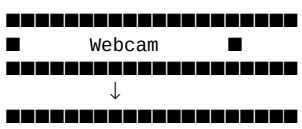
```
Hand starts at center
        ↓
Moves toward left
        ↓
Movement detected
        ↓
SWIPE LEFT
```

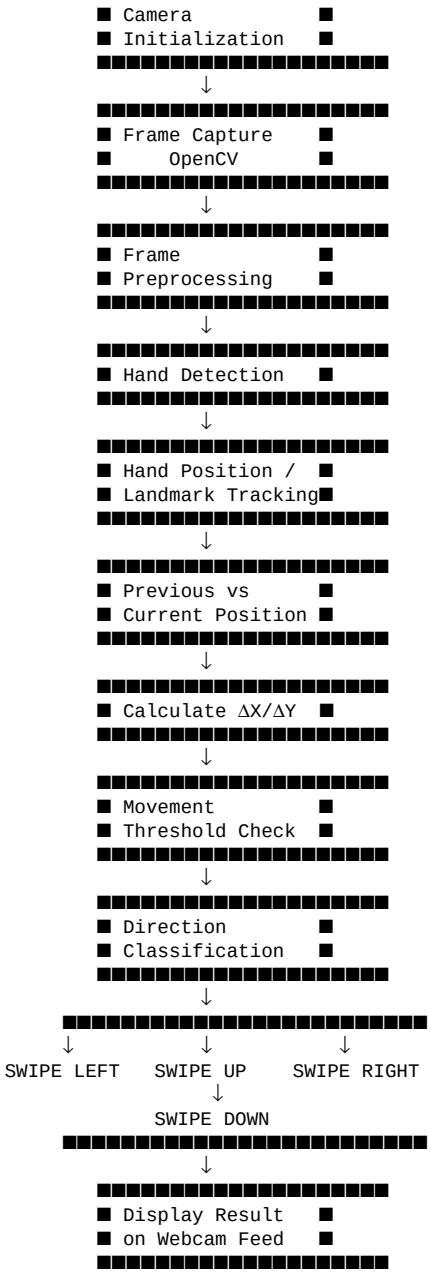
The project must implement the **second approach**.

---

5. HIGH-LEVEL SYSTEM ARCHITECTURE

Create a system architecture similar to:





Explain every component.

---

6. TECHNOLOGIES AND LIBRARIES

Use basic Python technologies.

Required

• Python

• OpenCV (`cv2`)

• NumPy

Optional

• MediaPipe for hand landmark detection/tracking

Explain whether MediaPipe is useful for this project.

If MediaPipe is used, explain that it can provide stable hand landmarks whose positions can be tracked across frames.

Do not make deep learning training necessary.

---

7. HAND DETECTION

Explain possible approaches.

Option A — OpenCV + NumPy

Use techniques such as:

```
Frame
 ↓
Preprocessing
 ↓
Segmentation / Thresholding
 ↓
Contour Detection
 ↓
Hand Bounding Box
 ↓
Centroid
```

The centroid can then be tracked between frames.

Option B — MediaPipe

```
Frame
 ↓
Hand Detection
 ↓
21 Hand Landmarks
 ↓
Select useful landmark
 ↓
Track landmark position
 ↓
Compare positions across frames
```

Explain which approach is better for a beginner class project.

---

8. HAND TRACKING

Explain that hand tracking is important because the system needs to know:

```
Where was the hand?
        ↓
Where is the hand now?
        ↓
How far did it move?
        ↓
Which direction did it move?
```

For example:

```
Time T1:
(x1, y1)

Time T2:
```

```
(x2, y2)

Time T3:
(x3, y3)
```

The system uses these positions to determine movement.

---

9. MOVEMENT CALCULATION

Explain:

```
ΔX = Current X - Previous X

ΔY = Current Y - Previous Y
```

Then compare the magnitude:

```
|ΔX| vs |ΔY|
```

Horizontal movement

If:

```
|ΔX| > |ΔY|
```

then movement is primarily horizontal.

```
ΔX > threshold
→ SWIPE RIGHT

ΔX < -threshold
→ SWIPE LEFT
```

Vertical movement

If:

```
|ΔY| > |ΔX|
```

then movement is primarily vertical.

```
ΔY > threshold
→ SWIPE DOWN

ΔY < -threshold
→ SWIPE UP
```

Explain that image coordinates usually increase downward, which is why the Y direction is interpreted accordingly.

---

10. MOVEMENT THRESHOLD

A small movement should not immediately trigger a swipe.

Explain the need for a **movement threshold**.

Example:

```
Small movement:
ΔX = 3
→ Ignore

Larger movement:
```

```
ΔX = 80
→ Possible SWIPE RIGHT
```

The threshold should be configurable.

For example:

```
MOVEMENT_THRESHOLD = configurable value
```

Do not assume a single fixed threshold is universally correct. Explain that it may depend on:

• Camera resolution

• Distance from camera

• Hand size

• Frame rate

---

11. SWIPE CONFIRMATION

To reduce false detections, explain a simple confirmation mechanism.

For example:

```
Frame 1 → position recorded
Frame 2 → movement detected
Frame 3 → movement continues
Frame 4 → threshold exceeded
              ↓
        SWIPE RIGHT
```

Possible rules:

• Minimum movement distance

• Minimum number of frames

• Dominant movement direction

• Optional cooldown after a swipe

Keep the implementation simple.

---

12. SWIPE COOLDOWN

Explain why a cooldown may be useful.

Without cooldown:

```
RIGHT RIGHT RIGHT RIGHT RIGHT
```

could be detected as multiple swipes from one continuous movement.

A simple cooldown can be:

```
SWIPE RIGHT
      ↓
Wait briefly
      ↓
Reset tracking
      ↓
Detect next swipe
```

Include this as a simple optional feature.

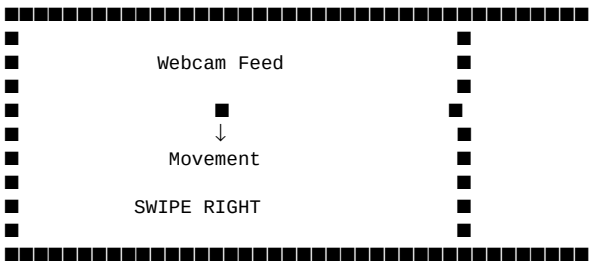
---

13. OUTPUT

The application should display:

```
SWIPE LEFT
SWIPE RIGHT
SWIPE UP
SWIPE DOWN
```

Example:



Also optionally display:

```
Hand Detected
Tracking...
```

when a swipe has not yet been completed.

---

14. ERROR HANDLING

Include a basic error-handling system.

Webcam Error

If the webcam cannot be opened:

```
Error: Could not open webcam.
```

Safely terminate.

Frame Error

If a frame cannot be read:

```
Warning: Frame could not be read.
```

Skip/retry or terminate after repeated failures.

No Hand Detected

This is a normal condition, not a system error.

Display:

```
No Hand Detected
```

Reset or pause tracking appropriately.

Hand Lost During Swipe

If the hand disappears while moving:

```
Tracking Lost
```

Reset the current movement calculation.

Invalid Coordinates

If the detector returns invalid coordinates:

```
Unknown / Tracking Lost
```

Do not crash.

Multiple Hands

Either:

• Track the largest/primary hand, or

• Display `Multiple Hands Detected`

Recommend the simplest approach.

Unexpected Runtime Error

Use basic exception handling where appropriate.

User Exit

Press:

```
Q
```

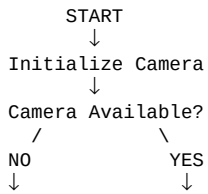
Then:

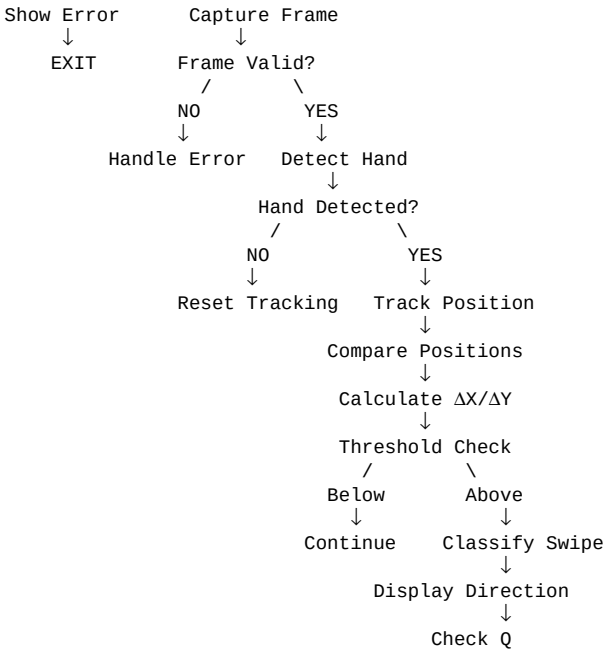
```
cap.release()
cv2.destroyAllWindows()
```

---

15. ERROR HANDLING FLOW

Create this diagram:





---

16. COMPLETE DATA FLOW

Show:

```
Webcam
  ↓
Video Frames
  ↓
Frame Validation
  ↓
Preprocessing
  ↓
Hand Detection
  ↓
Hand Position
  ↓
Previous Position
        +
Current Position
        ↓
Calculate ΔX / ΔY
        ↓
Threshold Check
        ↓
Swipe Classification
        ↓
SWIPE LEFT / RIGHT / UP / DOWN
        ↓
Display Result
```

---

17. PROJECT FOLDER STRUCTURE

Use a simple classroom structure:

```
hand_swipe_detection/
■
■■■ main.py
■■■ hand_detection.py
■■■ hand_tracking.py
■■■ swipe_detection.py
■■■ config.py
■■■ requirements.txt
■■■ README.md
```

```
■
■■■ utils/
■   ■■■ drawing.py
■
■■■ screenshots/
```

Explain each file.

`main.py`

Runs the main application loop.

`hand_detection.py`

Handles hand detection.

`hand_tracking.py`

Tracks hand/landmark position across frames.

`swipe_detection.py`

Calculates movement and determines swipe direction.

`config.py`

Stores thresholds and configurable settings.

`utils/drawing.py`

Handles bounding boxes, landmarks, movement arrows, and text.

`requirements.txt`

Contains required libraries.

`README.md`

Contains project documentation.

---

18. CONFIGURATION

Explain configurable values such as:

```
MOVEMENT_THRESHOLD
MIN_FRAMES
COOLDOWN_TIME
CAMERA_INDEX
FRAME_WIDTH
FRAME_HEIGHT
```

Do not hard-code everything into the main program.

---

19. OPTIONAL FREE API / FREE LLM INTEGRATION

The core application must work **without any API or LLM**.

However, optionally explain how a free API or free LLM could be added for demonstration purposes.

Possible optional feature:

```
Swipe Detected
      ↓
SWIPE RIGHT
      ↓
Optional LLM/API
      ↓
Generate a simple response/action
```

For example:

```
SWIPE RIGHT
     ↓
Optional AI interpretation
     ↓
"User swiped to the right"
```

The architecture should clearly separate:

Core System

```
Webcam → Hand Detection → Tracking → Swipe Detection
```

Optional AI Layer

```
Swipe Result → API/LLM → Optional Text/Action
```

Do not make an external API or LLM necessary for basic functionality.

Prefer free/local options where possible.

If recommending specific APIs or LLMs, verify their current free-tier availability before recommending them.

---

20. TESTING

Create simple test cases:

| Test Case | Input | Expected Output |

| --- | --- | --- |

| TC01 | Webcam unavailable | Camera Error |

| TC02 | Webcam available | Webcam Feed |

| TC03 | No hand | No Hand Detected |

| TC04 | Hand moves left | SWIPE LEFT |

| TC05 | Hand moves right | SWIPE RIGHT |

| TC06 | Hand moves up | SWIPE UP |

| TC07 | Hand moves down | SWIPE DOWN |

| TC08 | Very small movement | No Swipe |

| TC09 | Hand disappears | Tracking Lost |

| TC10 | Multiple hands | Primary Hand / Warning |

| TC11 | Invalid frame | Handle safely |

| TC12 | Press Q | Clean Exit |

---

21. SCOPE

In Scope

• Webcam input

• Single-hand detection

• Hand tracking

• Tracking position across frames

• Movement calculation

• ΔX / ΔY calculation

• Movement threshold

• Swipe direction classification

• LEFT

• RIGHT

• UP

• DOWN

• Real-time display

• Basic error handling

• Clean program exit

Out of Scope

• Face recognition

• Voice recognition

• Advanced multi-hand gesture recognition

• User authentication

• Database

• Networking

• Cloud infrastructure

• Mobile application

• Web application

• Production deployment

• Training a custom deep-learning model

Optional APIs/LLMs should remain outside the core functionality.

---

22. LIMITATIONS

Explain:

• Poor lighting may affect hand detection.

• Complex backgrounds may affect detection.

• Fast movement may cause missed detection.

• Very small movements may not trigger a swipe.

• Threshold values may need adjustment.

• Camera resolution and frame rate affect performance.

• Hand tracking can temporarily be lost.

• Multiple hands can create ambiguity.

• Basic tracking is not the same as advanced gesture recognition.

---

23. FUTURE IMPROVEMENTS

Mention as future scope:

• More gesture types

• Circle gestures

• Finger-count gestures

• Multiple-hand tracking

• Better tracking algorithms

• Gesture smoothing

• Machine-learning-based gesture classification

• GUI

• Gesture-controlled applications

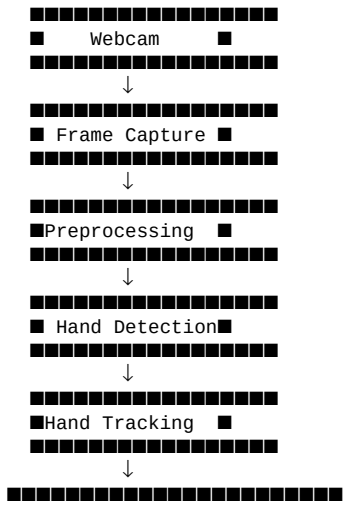
• Optional AI/LLM integration

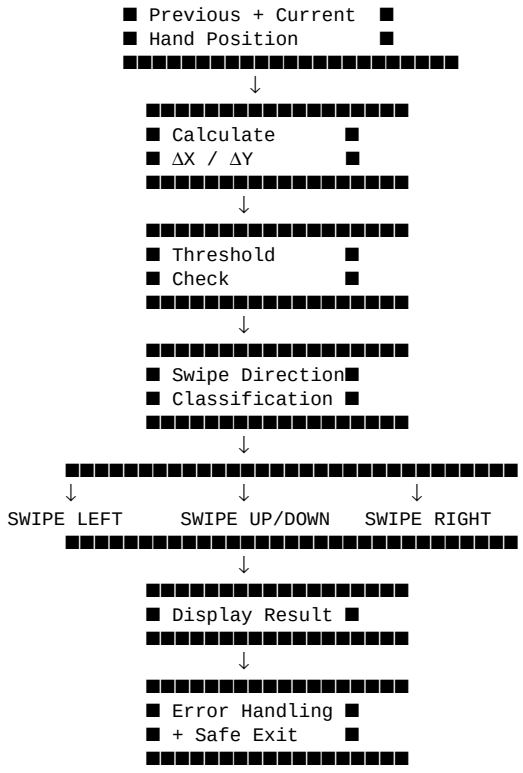
• Custom gesture training

---

24. FINAL ARCHITECTURE

End with a concise architecture:





FINAL INSTRUCTIONS

The generated architecture must be:

• Simple

• Beginner-friendly

• Practical

• Easy to implement

• Easy to explain during a class presentation

• Based primarily on Python + OpenCV + NumPy

• Optionally compatible with MediaPipe

• Focused on **hand movement/swipe detection**, NOT static hand position

• Based on comparing hand positions across multiple frames

• Include ΔX / ΔY movement calculation

• Include movement thresholds

• Include basic error handling

• Include testing

• Include a simple folder structure

• Include optional API/LLM integration only as an additional feature

Do not over-engineer the project.

The core functionality must work locally without an internet connection or external API.

The final result should look like a **properly structured computer vision class project**, not an enterprise software system.